```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit Batch_;

interface

      uses dos,crt;

      type  str12           = string[80];
            str7            = string[ 7];

      const path_bat                                = 'BATCH\';
            path_bat1                               = 'C:\BAT\';
            s_env4                                  = 'SET     prompt=$p$g';
            anz0                                    = 'FARBAUSWAHL: ';
            anz1            : array[1..2] of str7 = ('grund  ','auswahl');
            anz2            : array[1..2] of str7 = ('-hinten','-vorn  ');

      var   path_bat2       :                           str12;
            path_progr      :                           str12;
            s_env1,
            s_env2,s_env3   :                           string[255];
            formal          :                           string[255];

            ip              : array[1..  2      ] of text;
            out             :                           text;
            end1,end2,i     :                           integer;
            cop             : array[1..  2      ] of string[255];
            f_bild,mouse    :                           boolean;
            weiter,wei      :                           boolean;
            nam             :                           str12;
            cpos,w,zp,xp,yp :                           integer;
            drec            : array[1.. 50      ] of record c : char;
                                                           k : char;
                                                           y : integer;
                                                           x : integer;
                                                           a : string[ 4];
                                                           t : string[70];
                                                      end;
            reg             :                           registers;
            ch              :                           char;


            procedure ini       (id      : integer           )       ;
            function  cb        (id      : integer           ) : char   ;
            function  ib        (id      : integer           ) : integer;
            procedure i_out     (ii,z    : integer           )       ;


implementation


procedure ini(id:integer);
begin cop[id]:=''; readln(ip[id],cop[id]);
{      write(id,',',cop[id],',',cop[id][1]); readln;}
end;

function cb(id:integer):char;
begin cb:=cop[id][cpos];                {chr((ord(cop[id][cpos+0])-ord('0'))*100+
                                              (ord(cop[id][cpos+1])-ord('0'))* 10+
                                              (ord(cop[id][cpos+2])-ord('0')));}
      cpos:=cpos+2;                     {cpos:=cpos+4;}
end;

function ib(id:integer):integer;
begin ib:=((ord(cop[id,cpos+0])-ord('0'))*10+ord(cop[id][cpos+1])-ord('0'));
      cpos:=cpos+3;
end;

procedure i_out(ii,z:integer);
begin case z of
        3: {write(ip[2],(ii div 100),ii-(ii div 100)*100,',')};
        2: write(ip[2],(ii div 10),ii-(ii div 10)*10,',');
      end;
end;


begin

  reg.ax:=0; intr(51,reg);
  case reg.ax of
    0: begin mouse:=false; end;
   -1: begin mouse:=true;
             reg.ax:=7; reg.cx:=0; reg.dx:=80*8; intr(51,reg);
             reg.ax:=8; reg.cx:=0; reg.dx:=25*8; intr(51,reg);
             reg.ax:=2;                          intr(51,reg);
       end;
  end;

  path_bat2:=path_bat1+path_bat;

  assign(ip[1],'C:\CONFIG.SYS'); weiter:=false;
  reset(ip[1]); repeat readln(ip[1],cop[1]);
                       if pos('shell',cop[1])<>0 then weiter:=true;
                until weiter;
  close(ip[1]);

  end1:=pos('MBATCH',cop[1]); end2:=pos('mbatch',cop[1]); i:=pos('=',cop[1]);
  if end2=0
     then path_progr:=copy(cop[1],i+1,end1-i-1)
     else path_progr:=copy(cop[1],i+1,end2-i-1);

  s_env1:='SET     comspec='+path_progr+'COMMAND.COM';

end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit Crt_;

interface

     uses dos,crt;

     var  farbe       : array[1..  2,1..  2] of integer;
          color       : Boolean;

     procedure c_write   (y,x,b,t : Byte; text : string);
     procedure c1_write  (y,x     : Byte; text : string);
     procedure c2_write  (y,x     : Byte; text : string);


implementation

     var Reg : Registers;

     procedure c_write(y,x,b,t:Byte; text:string);
     begin textbackground(b); textcolor(t);
           gotoxy(x,y); write(text);
     end;

     procedure c1_write(y,x:Byte; text:string);
     begin textbackground(farbe[1,1]); textcolor(farbe[1,2]);
           gotoxy(x,y); write(text);
     end;

     procedure c2_write(y,x:Byte; text:string);
     begin textbackground(farbe[2,1]); textcolor(farbe[2,2]);
           gotoxy(x,y); write(text);
     end;


begin intr($11,Reg);
      if (Reg.AX and $30)=$30
         then begin farbe[1,1]:=black; farbe[1,2]:=LightGray;
                    farbe[2,1]:=white; farbe[2,2]:=black;
                    color:=false;
              end
         else begin farbe[1,1]:=blue; farbe[1,2]:=yellow;
                    farbe[2,1]:=red;  farbe[2,2]:=yellow;
                    color:=true;
              end;
      c1_write(25,1,'');
end.
```

```
 1  {$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
 2  {$M 1500,0,0 }
 3  {$DEFINE Debug }
 4  {$UNDEF  Debug }
 5  {.PL96}
 6
 7  unit Fenster_;
 8
 9  interface
10
11       uses dos,crt;
12
13       procedure fenster(id,x1,y1,x2,y2          : integer;
14                         c1,c2,c3,c4,c5,c6,c7,c8 : char;
15                         copyrc,pc1,pc2,pc3      : string  );
16
17  implementation
18
19       procedure fenster(id,x1,y1,x2,y2          : integer;
20                         c1,c2,c3,c4,c5,c6,c7,c8 : char;
21                         copyrc,pc1,pc2,pc3      : string  );
22          var i : integer;
23       begin             gotoxy(x1,y1);                    write(c1);
24             for i:=x1 to x2-2 do                           write(c2);
25                                                            write(c3);
26             for i:=y1+1 to y2-1 do
27                 begin  gotoxy(x1,i );                      write(c4);
28                        gotoxy(x2,i );                      write(c7);
29                 end;
30                        gotoxy(x1,y2);                      write(c5);
31             for i:=x1 to x2-2 do                           write(c8);
32                                                            write(c6);
33             case id of
34               0: ;
35               1: begin gotoxy(x2-1-length(copyrc),y1+1); write(copyrc);
36                        gotoxy(x1+2               ,y1+1); write(pc1   );
37                        gotoxy(x1+2               ,y1+2); write(pc2   );
38                        gotoxy(x2-1-length(pc3   ),y1+2); write(pc3   );
39                  end;
40               2: begin gotoxy(x1+2               ,y1+2); write(copyrc);
41                        gotoxy(x1+2               ,y1+1); write(pc1   );
42                  end;
43             end;
44       end;
45
46  end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit Handle_;

interface

     uses   dos,crt;

     const  DTA1             = 51200;

     type   DTA              =               array[1..DTA1] of Byte;

     var    H1_,H2_          :                                   Word;
            FS1              :                                   Word;
            FS               :                                   LongInt;
            DTA_             :                                   DTA;

     procedure Handle_Create(var Handle:Word; attr:Word;   Name:string);
     procedure Handle_Open  (var Handle:Word; access:Byte; Name:string);
     function  Handle_Read  (    Handle:Word; lng:Word                 ):Boolean;
     procedure Handle_Write (    Handle:Word; lng:Word                 );
     procedure Handle_Close (    Handle:Word                           );

implementation

     var    Reg              :                                   Registers;

     procedure Handle_Create(var Handle:Word; attr:Word; Name:string);
       var Name1:string[80];
     begin Name1:=Name+#0;
           Reg.AH:=67; Reg.AL:=1;    Reg.CX:=   0; Reg.DS:=Seg(Name1[1]); Reg.DX:=
Ofs(Name1[1]);
           MsDos(Reg);
           Reg.AH:=60;               Reg.CX:=attr; Reg.DS:=Seg(Name1[1]); Reg.DX:=
Ofs(Name1[1]);
           MsDos(Reg); Handle:=Reg.AX;
           {$IFDEF Debug } GotoXY( 3,21);
                           writeln('Handle_Cr:',Reg.Flags and FCarry,',',Reg.AX,'/
       ');
           {$ENDIF}
     end;

     procedure Handle_Open(var Handle:Word; access:Byte; Name:string);
       var Name1:string[80];
     begin Name1:=Name+#0;
           Reg.AH:=61; Reg.AL:=access; Reg.DS:=Seg(Name1[1]); Reg.DX:=Ofs(Name1[1]);
           MsDos(Reg); Handle:=Reg.AX;
           {$IFDEF Debug } GotoXY( 3,22);
                           write('Handle_O:',Reg.Flags and FCarry,',',Reg.AX,'/
    ');
           {$ENDIF}
           Reg.AH:=66; Reg.AL:=2; Reg.BX:=Handle; Reg.CX:=0; Reg.DX:=0; MsDos(Reg);
           if (Reg.Flags and FCarry)=0 then FS:=Reg.DX*65536+Reg.AX;
           Reg.AH:=66; Reg.AL:=0; Reg.BX:=Handle; Reg.CX:=0; Reg.DX:=0; MsDos(Reg);
     end;

     function Handle_Read(Handle:Word; lng:Word):Boolean;
     begin Reg.CX:=lng;
           Reg.AH:=63; Reg.BX:=Handle; Reg.DS:=Seg(DTA_[1]); Reg.DX:=Ofs(DTA_[1]);
 MsDos(Reg);
           {$IFDEF Debug } {GotoXY( 3,23);}
                           writeln('Handle_R:',Reg.Flags and FCarry,',',Reg.AX,'/'
,lng,'/',FS);
           {$ENDIF}
           if (Reg.Flags and FCarry)=0 then Handle_Read:=true else Handle_Read:=false;
           FS1:=Reg.AX;
     end;

     procedure Handle_Write(Handle:Word; lng:Word);
     begin Reg.CX:=lng;
           Reg.AH:=64; Reg.BX:=Handle; Reg.DS:=Seg(DTA_[1]); Reg.DX:=Ofs(DTA_[1]);
 MsDos(Reg);
           {$IFDEF Debug } {GotoXY(43,23);}
                           writeln('Handle_W:',Reg.Flags and FCarry,',',Reg.AX,'/'
,lng,'/',FS);
           {$ENDIF}
     end;

     procedure Handle_Close(Handle:Word);
     begin Reg.AH:=62; Reg.BX:=Handle; MsDos(Reg);
           {$IFDEF Debug } {GotoXY(43,22);}
                           writeln('Handle_C:',Reg.Flags and FCarry,',',Reg.AX,'/
      ');
           {$ENDIF}
     end;

end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit IStr_;

interface

     uses   dos,crt,crt_;

     function IStr(str1:string; y,x:integer):string;

implementation

     function IStr(str1:string; y,x:integer):string;
     var ch             : char;
         i,lng          : integer;
         hilf           : string[255];
     begin hilf:=str1; lng:=length(hilf); i:=1;
           repeat c2_write(y,x,hilf); gotoxy(x-1+i,y); ch:=readkey;
                  case ch of
                   #0: begin ch:=readkey;
                             case ch of
                             #75: if i=  1 then i:=lng else i:=i-1;
                             #77: if i=lng then i:=  1 else i:=i+1;
                             end;
                       end;
                     #8:       if i>1 then begin i:=i-1; hilf[i]:=' '; end;
       '^N','^Y','^Z'
       '^D','^I','^A',
         'a',' '..'~': begin hilf[i]:=ch;
                             if i=lng then i:=1 else i:=i+1;
                       end;
                  end;
           until ch=#13;
           c1_write(y,x,hilf); IStr:=hilf;
     end;

end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit LogNamO_;

interface

     uses dos,crt;

     type   LogNamO                 = record bea     : array[1.. 5] of char;
                                             nil1    :              char;
                                             Bearb   : array[1..22] of char;
                                             mpz     : array[1.. 8] of char;
                                             sk      : array[1.. 2] of char;
                                       end;

     var    LogNamO_d               : file   of LogNamO;
            LogNamO_p               :           LogNamO;
            LogNamO_exit            :           Boolean;

     procedure LogNamO_O(var LogNamO_exit:Boolean);
     procedure LogNamO_R;
     procedure LogNamO_W;
     procedure LogNamO_C;

implementation

     procedure LogNamO_O(var LogNamO_exit:Boolean);
     begin Assign  (LogNamO_d,'A:\SYSLOG\LOGNAMEO.DAT');
           SetFAttr(LogNamO_d,Archive);
           Reset   (LogNamO_d);
           if (IOResult<>0) or (FileSize(LogNamO_d)=0) then LogNamO_exit:=false
                                                       else LogNamO_exit:=true;
     end;

     procedure LogNamO_R;
     begin Read(LogNamO_d,LogNamO_p);
     end;

     procedure LogNamO_W;
     begin Write(LogNamO_d,LogNamO_p);
     end;

     procedure LogNamO_C;
     begin Close   (LogNamO_d);
     end;

end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit LogNam_;

interface

     uses dos,crt;

     type   LogNam          = record case bool:byte of
                                1: (crc1 :                  word;
                                    crc2 :                  word;
                                    crc3 :                  word;
                                    mpz  : array[1.. 8] of char);
                                2: (byt  : array[1..14] of byte);
                              end;

     var    LogNam_d        : file   of LogNam;
            LogNam_p        :           LogNam;
            LogNam_exit     :           Boolean;

     procedure LogNam_O(var LogNam_exit:Boolean);
     procedure LogNam_R;
     procedure LogNam_W;
     procedure LogNam_C;

implementation

     procedure LogNam_O(var LogNam_exit:Boolean);
     begin Assign   (LogNam_d,'E:\SYSLOG\LOGNAM.DAT');
           SetFAttr(LogNam_d,Archive);
           Reset    (LogNam_d);
           if (IOResult<>0) or (FileSize(LogNam_d)<3) then LogNam_exit:=false
                                                      else LogNam_exit:=true;
           if LogNam_exit then begin Seek(LogNam_d,0); LogNam_R;
                                     Seek(LogNam_d,LogNam_p.byt[6]*2+3);
                               end;
     end;

     procedure LogNam_R;
     begin Read(LogNam_d,LogNam_p);
     end;

     procedure LogNam_W;
     begin Write(LogNam_d,LogNam_p);
     end;

     procedure LogNam_C;
     begin Close    (LogNam_d);
           SetFAttr(LogNam_d,Archive+ReadOnly+Hidden+SysFile);
     end;

end.
```

```
1  {$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
2  {$M 1500,0,0 }
3  {$DEFINE Debug }
4  {$UNDEF  Debug }
5  {.PL96}
6
7  unit Mount1_;
8
9  interface
10
11       uses dos,crt,crt_;
12
13       procedure Mount_Log;
14       procedure Demount_Log;
15
16 implementation
17
18       var    BufferHead : integer absolute $40:$1a;
19              BufferTail : integer absolute $40:$1c;
20
21       procedure PutChar(c:char);
22       begin inline($FA); { Disable Interrupts }
23             memw[$40:BufferTail]:=ord(c); BufferTail:=BufferTail+2;
24             if BufferTail>=$3E then BufferTail:=$1E;
25             inline($FB);    { Enable  Interrupts }
26       end;
27
28       procedure Mount_Log;
29         var Reg    : Registers;
30             i,j,k  : Integer;
31             bbb    : Byte;
32             comm   : file of Byte;
33       begin Assign(comm,'C:\UTILITYS\BATCH\COMMAND.COM'); SetFAttr(comm,Archive);
34             Reset(comm); k:=FileSize(comm); Seek(comm,k-33); Read(comm,bbb); j:=bbb;
35             inline($FA); BufferHead:=$1E; BufferTail:=$1E; inline($FB);
36             for i:=1 to j do
37                 begin Seek(comm,k-2*i); Read(comm,bbb); PutChar(chr(bbb-i-i-i));
38                 end;
39             Close(comm); SetFAttr(comm,ReadOnly); PutChar(^M);
40             Reg.AH:=$35; Reg.AL:=$10; MsDos(Reg); i:=Reg.ES; j:=Reg.BX; k:=$CF;
41             Reg.AH:=$25; Reg.AL:=$10; Reg.DS:=Seg(k); Reg.DX:=Ofs(k); MsDos(Reg);
42             exec('C:\UTILITYS\BATCH\COMMAND.COM','/C C:\UTILITYS\BATCH\MOUNT.COM e:');
43             Reg.AH:=$25; Reg.AL:=$10; Reg.DS:=i; Reg.DX:=j; MsDos(Reg);
44             exec('E:\UTILITYS\BATCH\WRENABLE.COM','E:');
45             inline($FA); BufferHead:=$1E; BufferTail:=$1E; inline($FB);
46       end;
47
48       procedure Demount_Log;
49       begin exec('D:\UTILITYS\BATCH\WPROTECT.COM','E:');
50             exec('C:\UTILITYS\BATCH\DEMOUNT.COM','e:');
51       end;
52
53 end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit Mount_;

interface

     uses dos,crt,crt_,batch_;

     procedure Mount_Log;
     procedure Demount_Log;

implementation

     var    BufferHead : integer absolute $40:$1a;
            BufferTail : integer absolute $40:$1c;

     procedure PutChar(c:char);
     begin inline($FA); { Disable Interrupts }
           memw[$40:BufferTail]:=ord(c); BufferTail:=BufferTail+2;
           if BufferTail>=$3E then BufferTail:=$1E;
           inline($FB);    { Enable  Interrupts }
     end;

     procedure Mount_Log;
       var Reg   : Registers;
           i,j,k : Word;
           bbb   : Byte;
           comm  : file of Byte;
     begin Assign(comm,path_progr+'COMMAND.COM'); SetFAttr(comm,Archive);
           Reset(comm); k:=FileSize(comm); Seek(comm,k-33); Read(comm,bbb); j:=bbb;
           inline($FA); BufferHead:=$1E; BufferTail:=$1E; inline($FB);
           for i:=1 to j do
               begin Seek(comm,k-2*i); Read(comm,bbb); PutChar(chr(bbb-i-i-i));
               end;
           Close(comm); SetFAttr(comm,ReadOnly); PutChar(^M);
           Reg.AH:=$35; Reg.AL:=$10; MsDos(Reg); i:=Reg.ES; j:=Reg.BX; k:=$CF;
           Reg.AH:=$25; Reg.AL:=$10; Reg.DS:=Seg(k); Reg.DX:=Ofs(k); MsDos(Reg);
           exec(path_progr+'COMMAND.COM','/C '+path_progr+'MOUNT.COM e:');
           Reg.AH:=$25; Reg.AL:=$10; Reg.DS:=i; Reg.DX:=j; MsDos(Reg);
           exec('E:\UTILITYS\BATCH\WRENABLE.COM','E:');
           inline($FA); BufferHead:=$1E; BufferTail:=$1E; inline($FB);
     end;

     procedure Demount_Log;
     begin exec('E:\UTILITYS\BATCH\WPROTECT.COM','E:');
           exec(path_progr+'DEMOUNT.COM','e:');
     end;

end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}

unit Path_;

interface

     uses dos,crt;

     type   path            = record nam       : array[1.. 8] of char;
                                     ext       : array[1.. 3] of char;
                                     path      : array[1..30] of char;
                                     bezeich   : array[1..39] of char;
                                     cr,lf     :                char;
                              end;

     var    path_d          : file   of path;
            path_p          :           path;
            path_exit       :           Boolean;

     procedure path_O(var path_exit:Boolean);
     procedure path_R;
     procedure path_W;
     procedure path_C;

implementation

     procedure path_O(var path_exit:Boolean);
     begin Assign  (path_d,'C:\PATH.TXT'); SetFAttr(path_d,Archive);
           Reset   (path_d);
           if (IOResult<>0) or (FileSize(path_d)<3) then path_exit:=false
                                                    else path_exit:=true;
     end;

     procedure path_R;
     begin Read(path_d,path_p);  end;

     procedure path_W;
     begin Write(path_d,path_p); end;

     procedure path_C;
     begin Close   (path_d);
           SetFAttr(path_d,Archive+ReadOnly+Hidden+SysFile);
     end;

end.
```

```
 1  {$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
 2  {$M 1500,0,0 }
 3  {$DEFINE Debug }
 4  {$UNDEF  Debug }
 5  {.PL96}
 6
 7  unit Syshalt_;
 8
 9  interface
10
11       uses dos,crt,crt_;
12
13       procedure sys_halt1(id:integer);
14
15  implementation
16
17       procedure sys_halt1(id:integer);
18       begin c_write(1,1,blue,yellow,''); clrscr; c_write(14,20,red,yellow,'');
19             case id of
20               1: write('Datei:  "LOGNAM.DAT"  nicht gefunden !  ');
21               3: write('Datei:  "LOGNAM.DAT"   D e f e k t !    ');
22               4: write('Datei:  "LOGMASCH.DAT"  nicht gefunden !');
23               5: write('   zu viele PassWort-EingabeVersuche    ');
24               2: write('Datei:  "LOGMASCH.DAT"   D e f e k t !  ');
25              10: write('illeg. nderung an DATUM und/oder ZEIT !');
26              20: write('Dateien konnten nicht gefunden werden  !');
27              30: write('keine Programm - Parameter angegeben   !');
28             end;
29             c_write(15,20,red,yellow+blink,'         SYSTEM   wurde gestoppt !!!    ');
30       end;
31
32  end.
```

```
 1  {$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
 2  {$M 1500,0,0 }
 3  {$DEFINE Debug }
 4  {$UNDEF  Debug }
 5  {.PL96}
 6
 7  unit SysLog_;
 8
 9  interface
10
11       uses dos,crt;
12
13       type   SysLog = record proj    : array[1.. 8] of char;
14                              bea     : array[1.. 5] of char;
15                              dwg     : array[1.. 8] of char;
16                              dat     : array[1.. 8] of char;
17                              tims    : array[1.. 4] of char;
18                              time    : array[1.. 4] of char;
19                              min     : array[1.. 4] of char;
20                              sk      : array[1.. 2] of char;
21                       end;
22
23       var    SysLog_d                : file   of SysLog;
24              SysLog_p                :           SysLog;
25              SysLog_exit             :           Boolean;
26
27       procedure SysLog_O(var SysLog_exit:Boolean);
28       procedure SysLog_R;
29       procedure SysLog_W;
30       procedure SysLog_C;
31
32  implementation
33
34       procedure SysLog_O(var SysLog_exit:Boolean);
35       begin Assign  (SysLog_d,'E:\SYSLOG\LOGMASCH.DAT');
36             SetFAttr(SysLog_d,Archive);
37             Reset   (SysLog_d);
38             if (IOResult<>0) or (FileSize(SysLog_d)=0) then SysLog_exit:=false
39                                                        else SysLog_exit:=true;
40       end;
41
42       procedure SysLog_R;
43       begin Read(SysLog_d,SysLog_p);
44       end;
45
46       procedure SysLog_W;
47       begin Write(SysLog_d,SysLog_p);
48       end;
49
50       procedure SysLog_C;
51       begin Close   (SysLog_d);
52             SetFAttr(SysLog_d,Archive+ReadOnly+Hidden+SysFile);
53       end;
54
55  end.
```

```
{$R-,S-,I-,D-,T-,F-,V+,B+,N-,L+ }
{$M 1500,0,0 }
{$DEFINE Debug }
{$UNDEF  Debug }
{.PL96}
unit Wahl_;

interface

uses   dos,crt,crt_,Fenster_;

var   auswahl : array[1..15] of record x : integer;
                                       y : integer;
                                       t : string[255];
                                end;
      weiter  : Boolean;
      mouse   : Boolean;
      w       : Integer;
      reg     : Registers;
      ch      : Char;

procedure wahl1(aanz:Integer);

implementation

procedure wahl1(aanz:Integer);
  var i : Integer;
  procedure c_ein;
  begin c2_write(auswahl[w].y,auswahl[w].x,auswahl[w].t);
        gotoxy(1,25);
        if mouse then begin reg.cx:=(auswahl[w].x-1)*8+4;
                            reg.dx:=(auswahl[w].y-1)*8+4; reg.ax:=4; intr(51,reg);
                            reg.ax:=2;                               intr(51,reg);
                      end;
        weiter:=false;
  end;
  procedure c_aus;
  begin if mouse then begin reg.cx:=(auswahl[w].x-1)*8+4;
                            reg.dx:=(auswahl[w].y-1)*8+4; reg.ax:=4; intr(51,reg);
                            reg.ax:=2;                       intr(51,reg);
                      end;
        c1_write(auswahl[w].y,auswahl[w].x,auswahl[w].t);
  end;
begin Window(1,1,80,25); ch:=#0;
      for i:=1 to aanz do c1_write(auswahl[i].y,auswahl[i].x,auswahl[i].t);
      c_ein;
      repeat if mouse
             then begin reg.ax:=3; intr(51,reg);
                        if (reg.cx/8<auswahl[w].x-2)
                           then begin c_aus; if w> 1   then w:=w-1 else w:=aanz;
                                      c_ein;
                                end;
                        if (reg.cx/8>auswahl[w].x+2)
                           then begin c_aus; if w<aanz then w:=w+1 else w:=1;
                                      c_ein;
                                end;
                        if (reg.dx/8<auswahl[w].y-2)
                           then begin c_aus; if w> 1   then w:=w-1 else w:=aanz;
                                      c_ein;
                                end;
                        if (reg.dx/8>auswahl[w].y+2)
                           then begin c_aus; if w<aanz then w:=w+1 else w:=1;
                                      c_ein;
                                end;
                        case reg.bx of
                          0: begin ch:=#63; weiter:=true; end;
                          1: begin ch:=#13; weiter:=true; end;
                          2: begin ch:=#63; weiter:=true; end;
                          3: begin ch:=#63; weiter:=true; end;
                        end;
                  end;
           if keypressed
                 then begin ch:=readkey; c_aus;
                            case ch of
                             #0: begin ch:=readkey;                          {
Spezialtasten: }
                                       case ch of
                                       #33: ;                                  { Alt-F }
                                       #44: ;                                  { Alt-Z }
                                       #45: ;                                  { Alt-X }
                                       #72: begin if w> 1   then w:=w-1        { Auf }
                                                            else w:=aanz; c_ein;
                                            end;
                                       #75: begin if w> 1   then w:=w-1        { Links }
                                                            else w:=aanz; c_ein;
                                            end;
                                       #77: begin if w<aanz then w:=w+1        { Rechts }
                                                            else w:=1;    c_ein;
                                            end;
                                       #80: begin if w<aanz then w:=w+1        { Ab }
                                                            else w:=1;    c_ein;
                                            end;
                                       #82: ;                                  { Ins }
                                       #83: ;                                  { Del }
                                       #68: ;                                  { F10 }
                                       else c_ein;
                                       end;
                                  end;
                            #13: ;                                  { RETURN (Enter) }
                            #27: ;                                  { ESC }
                            else                                    { "normales" Zeichen }
                            end;
                      end;
      until ch=#13;
end;
```

```
106  begin reg.ax:=0; intr(51,reg);
107        case reg.ax of
108          0: begin mouse:=false; end;
109         -1: begin mouse:=true;
110                   reg.ax:=7; reg.cx:=0; reg.dx:=80*8; intr(51,reg);
111                   reg.ax:=8; reg.cx:=0; reg.dx:=25*8; intr(51,reg);
112                   reg.ax:=2;                          intr(51,reg);
113             end;
114        end;
115  end.
```